Guía De Laboratorio 11

Sistema Inteligente de Apoyo Pedagógico y Gestión del Aprendizaje para Instituciones Educativas

INTEGRANTES – GRUPO 6:

Rivera Castañeda, Jesús Martín	- U21213646
Quispe Fernández, Alonso	- U20208103
Alanya Guevara, Ben Andre	- U21315933
Ninatanta Fernández, Renato Ventura	- U21205210

ASIGNATURA:

Curso Integrador II: Software

DOCENTE:

Barturen Huaman, Renzo Renato

SECCIÓN:

31738

Lima – Perú

Índice

1. Resultado y alcance del incremento 3
1.1 Producto mínimo de la semana 3
2. Correspondencia con el sílabo 3
3. Fundamentos de calidad de compatibilidad 3
3.1 Interoperabilidad y coexistencia 3
3.2 Verificación y validación 4
3.3 Requisito, criterio, oráculo y evidencia 4
3.4 Capas de prueba y herramientas 4
3.5 Métricas operativas del laboratorio 5
3.6 Criterios de salida 5
3.7 Uso controlado de inteligencia artificial 6
4. Arquitectura del incremento 6
5. Sesión 21: evaluación de interoperabilidad 7
5.1 Actividad 1 — Línea base y contrato externo 7
5.2 Actividad 2 — Matriz de compatibilidad ejecutable 7
5.3 Actividad 3 — Revisar o implementar el adaptador de notificaciones 8
5.4 Actividad 4 — Pruebas de contrato y resiliencia 8
5.5 Actividad 5 — Interpretar la interoperabilidad sin inflar la conclusión 9
6. Sesión 22: coexistencia, regresión y gate 9
6.1 Actividad 6 — Desplegar el escenario de coexistencia 9
6.2 Actividad 7 — Aislamiento y operación concurrente 9
6.3 Actividad 8 — Regresión funcional 10
6.4 Actividad 9 — Gate de compatibilidad y CI 10
6.5 Actividad 10 — Informe de compatibilidad 10
7. Informe de compatibilidad 10
8. Riesgos conectados al incremento 11
9. Entregables y estado de evidencia (E0–E11) 11
10. Checklist de cierre 11

1. Resultado y alcance del incremento

Este avance corresponde a la Semana 11 del Curso Integrador II: Software y continúa la Unidad 3. Evalúa la calidad de compatibilidad de ESTUD-IA según ISO/IEC 25010:2023, concentrada en dos subcaracterísticas: interoperabilidad (intercambiar información con otro sistema y usar lo intercambiado) y coexistencia (funcionar eficazmente compartiendo entorno y recursos con otras instancias, sin interferencia).

Se parte de la versión 1 desplegada y del gate funcional de la Semana 10. Se incorpora un servicio externo de notificaciones simulado (contrato OpenAPI/JSON Schema y WireMock) al que ESTUD-IA avisa cuando un Alumno registra un intento de ejercicio. Luego se despliegan dos instancias de ESTUD-IA en el mismo host Docker (versión base y versión candidata) con puertos, bases de datos, volúmenes y logs aislados.

1.1 Producto mínimo de la semana

- Contrato externo del proveedor de notificaciones (POST /v1/notifications) revisado antes de escribir pruebas.
- Matriz de compatibilidad ejecutable (JSON) con validador, igual mecanismo de la Semana 10.
- Adaptador de notificaciones en Laravel con timeout acotado, reintentos máximos y auditoría, que no revierte el intento ya confirmado.
- Pruebas de contrato y resiliencia INT-F01 a INT-F05 contra WireMock (éxito, duplicado, 400, 5xx con reintento y timeout).
- Escenario de coexistencia con Docker Compose: dos APIs Laravel, dos bases MySQL y el stub, con smoke concurrente.
- Regresión funcional de la Semana 10 y gate de compatibilidad local y en CI.
- Informe firmado con métricas obtenidas, defectos, límites y decisión técnica.

No se evalúan usabilidad, accesibilidad ni rendimiento: pertenecen a la Semana 12 y a la Unidad 4.

2. Correspondencia con el sílabo

Sesión	Tema del sílabo	Aplicación en ESTUD-IA
21	Calidad de software orientada a la compatibilidad según ISO 25010. Interoperabilidad.	Contrato externo, adaptador de notificaciones, pruebas de contrato y de resiliencia.
22	Calidad de software orientada a la compatibilidad según ISO 25010. Coexistencia.	Despliegue de dos instancias en el mismo host: aislamiento de puertos, base de datos, volúmenes y logs, y smoke concurrente.

3. Fundamentos de calidad de compatibilidad

3.1 Interoperabilidad y coexistencia

Subcaracterística	Pregunta aplicada a ESTUD-IA	Evidencia suficiente
Interoperabilidad	¿La aplicación envía y recibe información del servicio externo respetando el contrato y usando la respuesta?	Contrato validado, prueba de éxito, de idempotencia, errores 400/409/5xx/timeout y auditoría.
Coexistencia	¿Dos instancias de ESTUD-IA operan en el mismo host sin colisión de puertos, bases o volúmenes ni degradación indebida?	Compose con puertos y bases separados, healthchecks, smoke concurrente y aislamiento de datos.

La interoperabilidad no equivale a hacer una llamada HTTP: exige contrato, semántica, manejo de errores, idempotencia y evidencia de que la información se usa correctamente. La coexistencia no equivale a levantar dos contenedores: exige aislamiento de recursos y comprobar que una instancia no contamina a la otra.

3.2 Verificación y validación

Actividad	Pregunta	Ejemplo en ESTUD-IA
Verificación	¿El producto cumple una condición especificada?	El adaptador envía Idempotency-Key igual al código público del intento y recibe 202.
Validación	¿Permite cumplir la necesidad de uso prevista?	Un intento registrado genera una notificación externa o queda auditado como fallido sin romper la confirmación del Alumno.

3.3 Requisito, criterio, oráculo y evidencia

Elemento	Ejemplo
Requisito	RF-NOT-02: la aplicación no duplica notificaciones para un mismo intento.
Criterio	Dos envíos con el mismo intento producen una sola notificación aceptada; un 409 del proveedor se trata como duplicado aceptado.
Oráculo	Una llamada con 202 y otra con 409; una sola clave de idempotencia distinta en el stub; evento de auditoría NOTIFICACION_DUPLICADA_ACEPTADA.
Prueba	INT-F02 (la notificación duplicada es idempotente).
Evidencia	Reporte de PHPUnit, verificación de WireMock y consulta de auditoría.

3.4 Capas de prueba y herramientas

Capa	Herramienta	Qué demuestra	Qué no demuestra
Unidad	JUnit/AssertJ → PHPUnit	Reglas de mapeo y decisión del adaptador.	Contrato real ni red.
Contrato	Spring Boot + WireMock → Feature tests de Laravel contra WireMock real	Cabeceras, cuerpo, códigos y errores del proveedor simulado.	Semántica completa del proveedor real.
Coexistencia	Docker Compose, curl, psql y Bash → Compose, curl, cliente mysql y Bash	Aislamiento de puertos, base, volúmenes, logs y salud.	Rendimiento bajo carga ni alta disponibilidad.

Capa	Herramienta	Qué demuestra	Qué no demuestra
Regresión	Gate Semana 10 y Playwright	Que el flujo funcional principal no se rompió.	Cobertura exhaustiva.
IA supervisada	Asistente autorizado	Escenarios adicionales revisados por personas.	Decisión de calidad ni oráculo automático.

Nota técnica: Http::fake de Laravel sirve para pruebas unitarias del mapeo, pero no demuestra el contrato real; por eso las pruebas INT-F0x se ejecutan contra un WireMock real (contenedor local o servicio del runner de CI).

3.5 Métricas operativas del laboratorio

Operacionalización didáctica del proyecto; no son fórmulas normativas de ISO/IEC 25010.

Métrica	Cálculo	Lectura
Conformidad de contrato	Escenarios de contrato aprobados ÷ ejecutados × 100	Un escenario omitido no cubre el requisito.
Resiliencia de interoperabilidad	Escenarios de error, timeout e idempotencia aprobados ÷ ejecutados × 100	No premia reintentos infinitos ni silencios.
Aislamiento de coexistencia	Comprobaciones de puerto, base, volumen, log y salud aprobadas ÷ ejecutadas × 100	Una colisión bloquea la coexistencia.
Operación concurrente	Solicitudes exitosas en ambas instancias ÷ solicitudes concurrentes × 100	No mide capacidad; solo que no se interfieren.
Trazabilidad	Requisitos críticos con prueba ejecutada y evidencia ÷ requisitos críticos × 100	Un requisito sin ejecución queda no cubierto.
Regresión funcional	Casos críticos de la Semana 10 aprobados ÷ ejecutados × 100	La compatibilidad no debe romper la función.

3.6 Criterios de salida

Condición	Decisión
Un requisito crítico no tiene prueba ejecutada o aparece como skip	Bloquea el informe de compatibilidad.
El contrato externo no coincide en ruta, método, cabecera, cuerpo o código esperado	Bloquea la interoperabilidad afectada.
Un timeout o 5xx no se maneja y rompe el intento ya confirmado	Bloquea la versión hasta corregir y repetir.
Se duplica una notificación para el mismo intento	Bloquea la interoperabilidad.
Dos instancias comparten puerto, base de datos o volumen de datos	Bloquea la coexistencia.
El smoke concurrente falla o una instancia contamina a la otra	Bloquea la coexistencia.

Condición	Decisión
La regresión funcional de la Semana 10 falla	Bloquea la versión hasta corregir.
Una herramienta no puede ejecutarse	Marcar Bloqueado, conservar el diagnóstico y no atribuir un resultado.

3.7 Uso controlado de inteligencia artificial

La IA puede proponer escenarios de error, variantes de payload y casos de coexistencia, pero no conoce el contrato verdadero ni decide qué comportamiento es correcto. Su salida se trata como cambio no confiable hasta que una persona revisa el plan, el diff y la ejecución.

Riesgo	Control obligatorio
Endpoint, cabecera o código inventado	Contrastar con el contrato OpenAPI del proveedor y con la matriz.
Prueba que pasa sin observar el resultado	Exigir aserción sobre request, response, auditoría o estado persistente.
Reparación que oculta el defecto	Rechazar skip, force, esperas arbitrarias y expectativas más laxas.
Exposición de datos o secretos	Solo fixtures sintéticos; no enviar tokens de Sanctum, APP_KEY, .env ni logs.
Cambio no reproducible	Guardar brief, plan y diff; versionar solo código revisado y ejecutado.

4. Arquitectura del incremento

La guía oficial usa Spring Boot, Failsafe, WireMock y Compose con PostgreSQL. En ESTUD-IA se conserva la intención de cada pieza con el stack real.

Archivo de la guía	Equivalente en ESTUD-IA	Responsabilidad
quality/week11/requirements-compatibility.json	Igual, con requisitos RF-NOT, RF-COE y RF-REG	Declarar requisitos con criterios observables.
scripts/validate-compatibility-matrix.mjs	Igual	Detectar requisitos sin ID, criterio, subcaracterística o prueba.
contracts/notification-provider.openapi.yaml	Igual	Contrato del servicio externo simulado.
NotificationClientIT.java	backend/tests/Feature/Integration/NotificationClientTest.php	Probar contrato, idempotencia, errores y

Archivo de la guía	Equivalente en ESTUD-IA	Responsabilidad
		timeout contra WireMock.
NotificationClient (Spring)	app/Services/NotificationClient.php con el cliente Http de Laravel (timeout y retry acotados)	Adaptador del proveedor externo.
deploy/compose.week11.yaml	Mismo archivo con dos servicios Laravel, dos MySQL y WireMock	Desplegar las dos instancias y el stub.
scripts/coexistence-smoke.sh y week11-compatibility-gate.sh	Mismos scripts adaptados a /up de Laravel y cliente mysql	Verificar aislamiento y ejecutar el gate.
actuator/health	Ruta /up de Laravel (o una ruta /healthz propia)	Salud de cada instancia.

5. Sesión 21: evaluación de interoperabilidad

5.1 Actividad 1 — Línea base y contrato externo

- Registrar el commit de la versión 1 y ejecutar el gate de la Semana 10 (o como mínimo php artisan test, npm test, npm run build y el smoke local).
- Crear quality/week11, evidence/week11, deploy y scripts, y copiar el contrato externo.
- Revisar que el contrato declare POST /v1/notifications, cabeceras X-Correlation-ID e Idempotency-Key, cuerpo JSON y respuestas 202, 400, 409 y 500.

Si una prueba antigua falla, no se elimina ni se atribuye a la Semana 11 sin reproducir la causa. Evidencia E1: commit, versiones, salida del gate previo, contrato revisado y bloqueos iniciales.

5.2 Actividad 2 — Matriz de compatibilidad ejecutable

La matriz contiene solo comportamiento comprometido. El validador debe terminar con código cero e informar tres subcaracterísticas (interoperabilidad, coexistencia y regresión) y MATRIX OK; esto solo valida estructura y trazabilidad, no que las pruebas pasen.

ID	Subcaracterística	Crítico	Criterio observable	Pruebas
RF-NOT-01	interoperability	Sí	El adaptador hace POST /v1/notifications con X-Correlation-ID, Idempotency-Key y cuerpo válido; 202 equivale a estado SENT.	INT-F01
RF-NOT-02	interoperability	Sí	No se duplica la notificación de un intento: 409 del proveedor se trata como duplicado aceptado, sin reintento.	INT-F02
RF-NOT-03	interoperability	Sí	400 equivale a FAILED_VALIDATION sin reintento; 5xx o timeout reintentan como máximo 2 veces y luego FAILED, con el intento confirmado y evento NOTIFICACION_FALLIDA.	INT-F03, INT-F04, INT-F05

ID	Subcaracterística	Crítico	Criterio observable	Pruebas
RF-COE-01	coexistence	Sí	Dos instancias usan puertos, bases, volúmenes y nombres de contenedor distintos.	COE-F01
RF-COE-02	coexistence	Sí	Un intento creado en la instancia A no aparece en la base de B y viceversa; ambas responden en simultáneo.	COE-F02
RF-REG-01	regression	Sí	El flujo crítico de la Semana 10 (WEB-F02) sigue aprobando.	WEB-F02

Evidencia E2: matriz JSON y salida del validador.

5.3 Actividad 3 — Revisar o implementar el adaptador de notificaciones

Aspecto	Requisito del adaptador en Laravel
URL base	Tomada de NOTIFICATION_BASE_URL en el .env (nunca fija en el código).
Timeouts	Conexión y lectura acotados (por ejemplo 500 a 2000 ms en pruebas) con Http::timeout().
Cabeceras	X-Correlation-ID con la correlación de la Semana 7 e Idempotency-Key igual al código público del intento.
Cuerpo JSON	Identificador público del intento, del alumno y del ejercicio, código del curso, estado y fecha de ocurrencia; sin correo ni datos personales.
202	Estado SENT.
409	DUPLICATE_ACCEPTED, sin reintento.
400	FAILED_VALIDATION, sin reintento.
5xx o timeout	Reintentos acotados (máximo 2) y luego FAILED.
Efecto sobre el intento	El intento confirmado no se revierte; se registra el evento de auditoría. La llamada ocurre después del commit (afterCommit o un job en cola) para no bloquear la respuesta al Alumno.

Evidencia E3: código revisado, configuración de timeout, política de reintentos y auditoría asociada.

5.4 Actividad 4 — Pruebas de contrato y resiliencia

Caso	Oráculo principal	Aporte
INT-F01	POST a /v1/notifications con cabeceras y cuerpo válidos; 202 equivale a SENT.	Conformidad de contrato.
INT-F02	Dos envíos del mismo intento; 409 equivale a DUPLICATE_ACCEPTED; sin duplicar notificación.	Idempotencia.
INT-F03	400 equivale a FAILED_VALIDATION; una sola llamada; auditoría.	Error de entrada.

Caso	Oráculo principal	Aporte
INT-F04	500 y luego 202 equivale a SENT; máximo 3 intentos totales.	Reintento acotado.
INT-F05	Timeout equivale a FAILED; el intento sigue confirmado; auditoría NOTIFICACION_FALLIDA.	Degradación controlada.

Evidencia E4: reporte de PHPUnit con los cinco casos sin omisiones.

5.5 Actividad 5 — Interpretar la interoperabilidad sin inflar la conclusión

- Contar los requisitos de interoperabilidad y los que tienen al menos una prueba aprobada.
- Revisar los oráculos críticos: contrato, idempotencia, error 400, 5xx o timeout y auditoría.
- Registrar cualquier diferencia entre contrato e implementación; no se cambia el contrato para describir un defecto si la regla de negocio exige otro resultado.
- Redactar cinco a ocho líneas con versión, alcance, resultados y límites.

Evidencia E5: métricas de interoperabilidad y conclusión acotada.

6. Sesión 22: coexistencia, regresión y gate

La segunda sesión despliega dos instancias de ESTUD-IA en el mismo host y verifica que no se interfieren, siempre en entorno local o de pruebas con datos sintéticos.

6.1 Actividad 6 — Desplegar el escenario de coexistencia

Recurso	Instancia A (base)	Instancia B (candidata)	Compartido
API Laravel	Puerto 8088	Puerto 8098	No
Base MySQL	estudia_a (volumen propio)	estudia_b (volumen propio)	No
Variables de entorno	env propio, APP_KEY propia	env propio, APP_KEY propia	No
Prefijo de caché y limitador	Propio	Propio	No
Nombre de contenedor y logs	Con INSTANCE_ID=A	Con INSTANCE_ID=B	No
Stub de notificaciones (WireMock)	Puerto 9099	Puerto 9099	Sí, por diseño
Red Docker	Red compartida	Red compartida	Sí, por diseño

Se levanta con docker compose y se comprueba la salud de ambas APIs (/up) y del stub. Si una instancia no arranca, se revisan los logs antes de modificar pruebas. Evidencia E6: compose ps, healthchecks y logs iniciales.

6.2 Actividad 7 — Aislamiento y operación concurrente

- El smoke de coexistencia comprueba puertos distintos, bases distintas, volúmenes no compartidos, logs con INSTANCE_ID, salud y solicitudes concurrentes a ambas instancias.
- Resultado esperado: COEXISTENCE_SMOKE=PASS; un intento creado en A no aparece en la base de B y viceversa.
- La contraseña sintética se introduce por variable de entorno. No se borran datos de producción para reutilizar fixtures: se prepara un entorno sintético nuevo.

Evidencia E7: salida completa del smoke, consultas de aislamiento y logs por instancia.

6.3 Actividad 8 — Regresión funcional

Se confirma que la integración externa y la coexistencia no rompieron el flujo crítico: se ejecuta el gate funcional de la Semana 10 contra la instancia A o el caso WEB-F02 una sola vez sobre un ejercicio sintético libre. No se usa test.skip: si falta el ejercicio o la contraseña, el caso falla con un diagnóstico explícito; si el fixture ya fue consumido, se prepara otro entorno sintético o se registra Bloqueado. Evidencia E8: reporte de regresión sin casos omitidos.

6.4 Actividad 9 — Gate de compatibilidad y CI

Una sola secuencia que se detiene ante la primera condición no satisfecha: validar la matriz, ejecutar las pruebas de contrato, levantar la coexistencia, correr el smoke y la regresión. Se repite en un runner limpio con un workflow de GitHub Actions de permisos mínimos, con servicios MySQL y WireMock, publicando reportes como artefactos. Un gate verde significa que las condiciones codificadas pasaron en esa ejecución; no que el producto carezca de defectos. Evidencia E9: salida del gate local, ejecución CI y reportes.

6.5 Actividad 10 — Informe de compatibilidad

La tabla de la siguiente sección se completa con valores obtenidos, no estimados. Un enlace de evidencia vacío mantiene el requisito Pendiente.

7. Informe de compatibilidad

Indicador	Resultado	Evidencia	Interpretación
Conformidad de contrato	___ %	PHPUnit y WireMock	Escenarios de contrato aprobados.
Resiliencia de interoperabilidad	___ %	PHPUnit y auditoría	Errores, timeout e idempotencia manejados.
Aislamiento de coexistencia	___ %	Compose y smoke	Puertos, bases, volúmenes y logs aislados.
Operación concurrente	___ %	Smoke concurrente	Ambas instancias operan sin interferencia.
Regresión funcional	___ %	Gate Semana 10	Flujo crítico no roto.
Casos de IA	___ aceptados / ___ rechazados	Plan, decisiones y diff	Aporte revisado; no es métrica del producto.

Los defectos y bloqueos se describen por síntoma, condición, resultado real, resultado esperado y evidencia. El informe cierra con el alcance no evaluado y la decisión técnica: Aprobada, Condicionada o Bloqueada. Todo queda en blanco hasta ejecutar.

8. Riesgos conectados al incremento

Se retoman los riesgos R-01 a R-20 de los avances anteriores y se agregan dos propios de esta semana.

ID	Riesgo formulado	Disparador	Control / respuesta	Indicador
R-21	Si la llamada al proveedor de notificaciones se hace dentro de la petición del Alumno, un timeout o reintentos podrían retrasar o romper la confirmación del intento.	El POST de intento supera el timeout configurado cuando el stub responde 500 o no responde.	Invocar el adaptador después del commit o en un job en cola, con timeout y reintentos acotados; probarlo con INT-F04 e INT-F05.	SLI-02
R-22	Si las dos instancias comparten APP_KEY, prefijo de caché, volumen de almacenamiento o base de datos, una podría contaminar a la otra (tokens, limitador de login, datos).	El smoke de aislamiento encuentra datos de A en la base de B o un recurso compartido no previsto.	Un env y una APP_KEY por instancia, prefijos de caché distintos, bases y volúmenes separados, y el smoke de aislamiento como parte del gate.	DEL-01

9. Entregables y estado de evidencia (E0–E11)

ID	Entregable	Contenido mínimo	Estado
E0	Inicio controlado	Issue, rama feature/semana11-compatibilidad y commit base.	Pendiente
E1	Línea base y contrato	Commit, versiones, gate previo y contrato externo revisado.	Pendiente
E2	Matriz de compatibilidad	JSON de requisitos y salida MATRIX OK.	Pendiente
E3	Adaptador	Código, timeouts, política de reintentos y auditoría.	Pendiente
E4	Contrato y resiliencia	PHPUnit con INT-F01 a INT-F05 sin omisiones.	Pendiente
E5	Interpretación	Métricas de interoperabilidad y conclusión acotada.	Pendiente
E6	Escenario de coexistencia	Compose, healthchecks y logs iniciales.	Pendiente
E7	Smoke de coexistencia	Salida completa con COEXISTENCE_SMOKE=PASS.	Pendiente
E8	Regresión funcional	Reporte del gate de la Semana 10 sin omisiones.	Pendiente
E9	Gate y CI	Salida local, ejecución en runner limpio y artefactos.	Pendiente
E10	Informe	Informe firmado con versión, alcance, métricas, defectos, límites y decisión.	Pendiente
E11	Defensa	Presentación y Pull Request con revisión por pares.	Pendiente

10. Checklist de cierre

- El contrato externo se revisó antes de escribir las pruebas y no se modificó para describir un defecto.

- Cada requisito crítico tiene criterio, oráculo, prueba ejecutada y evidencia enlazada.
- INT-F01 a INT-F05 se ejecutaron contra un WireMock real, sin skip.
- Un timeout o 5xx del proveedor no revierte ni rompe el intento ya confirmado.
- Las dos instancias no comparten puerto, base, volumen, APP_KEY ni prefijo de caché.
- El smoke concurrente y la regresión funcional de la Semana 10 aprobaron.
- La IA no recibió secretos ni datos personales y ninguna reparación relajó el oráculo.
- El gate local y el workflow usan dependencias fijadas y reportes recuperables.
- El informe distingue resultados, límites y bloqueos; no afirma ausencia de defectos.
- Los riesgos R-21 y R-22 quedan vinculados al registro de riesgos de ESTUD-IA.